

# An Alternative Multi-Boot Architecture for the Raspberry Pi

## Introduction

First a **disclaimer**. This project is only beta and was something to occupy me during the winter months. I wouldn't touch this at all unless you are fairly familiar with how the Raspberry Pi boots.

I have tried to create a multi-boot system that has the following properties:-

- Boots **.img** files rather than physical partitions.
- All os images would use their own kernels and their own dtb structure (base,overlay,properties), and any provided initramfs.
- Images could be anywhere, on any disk or any partition
- An images partition could contain os images for multiple pi models. The ROOT partition of these images can even be shared across models where they are compatible.
- Possibility of extending the size of an image without hitting barriers due to a succeeding physical partition
- Be able to use a (slightly modified) version of the Rpi-Imager tool to create the .img files from the official source repository

## Limitations

There are some limitations to what this system will boot:-

- The kernel of the image must include a uefi stub as this project uses the uefi bootloaders created by the [Pi Firmware Task Force](#)
- 64 bit images only. This is a little limiting for the pi3b
- Image partitions can contain images for multiple devices but the single boot partition has to be device specific
- Booting of some of the more exotic images that I've tried needs image specific entities, maybe a specific dtb and/or more frequently a second small initramfs to overlay onto the one that the image provides.
- Most tested images comply with the non-initramfs boot process described below and do not use an initramfs overlay. However images marked with a "\*" require the use on an overlay initramfs in order to get ahead of the standard boot procedure to loop mount the image and make its partitions available.

## Tested images

- pi3b
    - Raspberry Pi OS (64-bit)Trixie
    - Raspberry PI OS Lite Trixie (64 bit)
    - Ubuntu Server 24.04.4 LTS (64 bit)
  - pi4
    - Raspberry Pi OS (64-bit)Trixie
    - Raspberry PI OS Lite Trixie (64 bit)
    - Ubuntu Server 25.10 (64 bit)
    - Ubuntu Workstation 26.04 (64 bit) \* - Note that Ubuntu has moved to a complex initramfs much like fedora with this version
    - Fedora-Workstation-Disk-43-1.6 \*
    - LibreELECT 12.2.1 \*
    - Lineage-23.2-20260128-UNOFFICIAL-KonstaKANG-rpi4 (includes booting to twrp as a seperate grub entry)
- \*

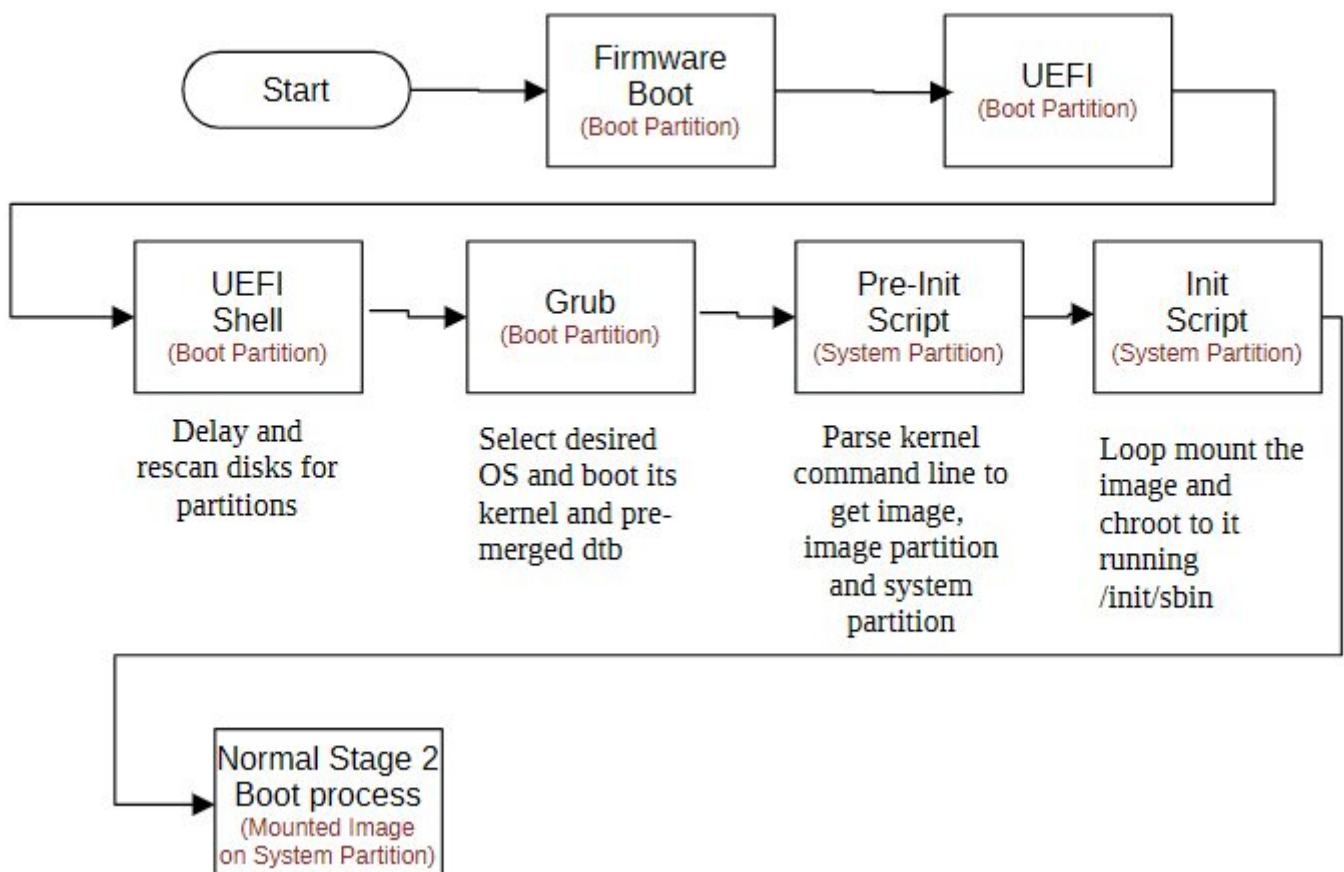
# Disk Structure

The disk structure is the following :-

1. A per Pi model **BOOT<Model>** (BOOTPI3,BOOTPI4) partition. This can be on a master drive if you have all the same hardware type or on, for example, its own USB key or microSD with a common drive for other partitions if you want to boot images for the Pi3B and Pi4 on the same master disk  
It contains the UEFI bootloader and the **grub** binary and **grub.cfg** which the multiboot system relies upon
2. A single systemwide **SYSTEM** partition. This contains the logic for the non-initramfs multiboot process
3. One or more **IMAGE** partitions on one or more disks. This contains the os images. Note that some of the utilites provided assume just one such partition but it isn't a hard limitation

## How it works (no or passive initramfs)

Essentially the system works by presenting the kernel with a false **Root** partition. The kernel runs its own PID1 routine, either directly or with an initramfs (again, only simple ones are supported in this model). It then chains through to the init process in the false Root partition, which loop mounts the image of the *real* Root partition (still in PID1), remounts the critical proc,sys and dev inside it, and chroots to the *real* init (usually /sbin/init) The image below describes the flow of this boot process in more detail.



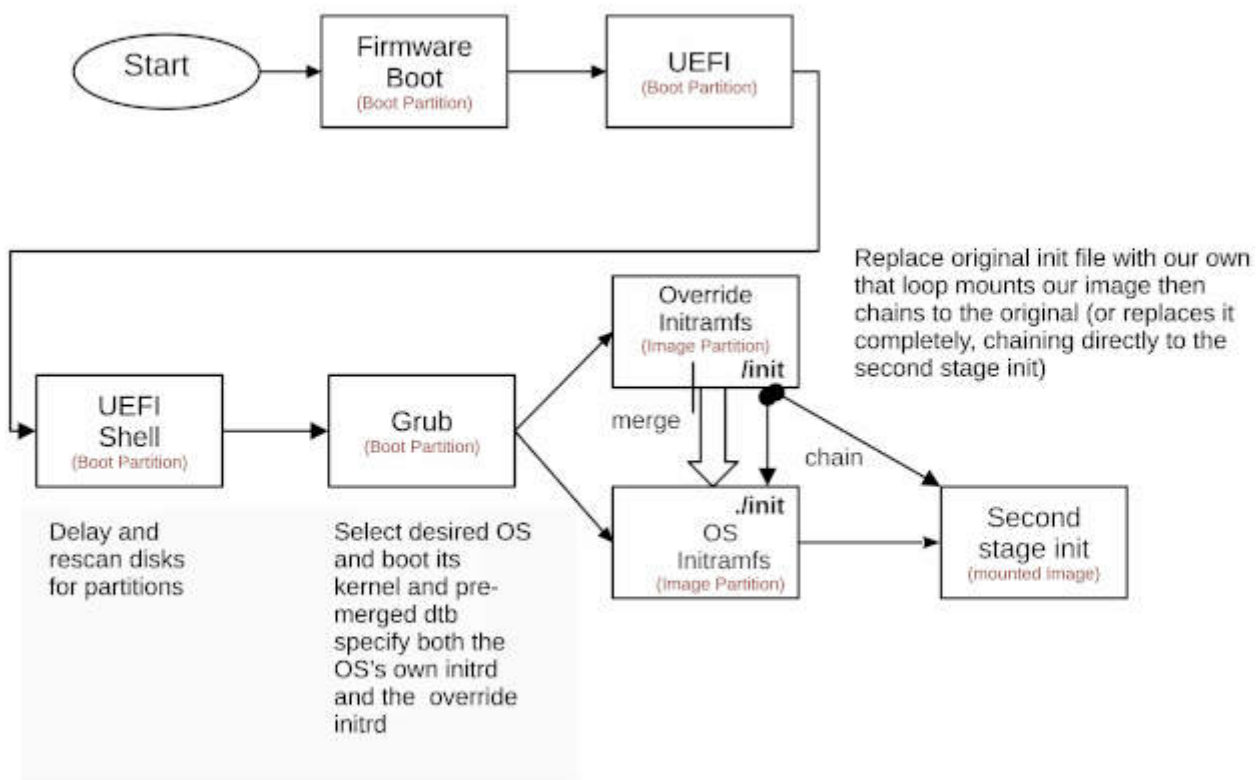
1. The Pi firmware loads the uefi firmware in the boot partition. Note that the (provided) config.txt file used here is minimal.

The dtb properties and any overlays along with overlay properties in the original image boot sector config.txt are used to create a pre-merged dtb file for grub. Some rare non-dtb properties can be set here, although they will be set across all images and in fact, very few survive the UEFI boot chain.

2. The uefi firmware loads [shellaa64.efi](#), renamed as BOOTAA64.EFI. The purpose of this is just to introduce a delay and then request the uefi software to rescan for disk partitions in order to accommodate slow disks. It may not be necessary, in which case you can rename grub.efi to BOOTAA64.EFI and take this stage out.
3. grub loads its config file with the list of bootable images. When an image is selected it scans for the system partition and the images partition containing the selected image. It obtains the PARTUUID's and UUID's for the target images partition and system partition, passing them as kernel parameters to the next stage.
4. The kernel is booted, along with any minimal non-active initramfs, with the system preinit script specified as the INIT parameter
5. The preinit script parses the kernel command line to obtain the UUIDs of the target and system partitions. It then does a chroot to a simulated, filebased, initrd environment specifying the init script of that environment as next step
6. The init script (using busybox) loop mounts the root partition of the specified img file, mounts the relevant proc and sys mounts inside it and chroot's to /sbin/init inside the img loop mount
7. Second stage boot continues as per normal

## How it works (active initramfs)

OS that use an active initramfs will have inside the initramfs a file at root level '**init**', which quite often is just a symlink to somewhere else. the init file contains or links to the stage 1 boot process, aimed at mounting and running the physical partitions of the image. It is necessary to identify the target image and loop mount it before that handover so the the subsequent first stage init can operate on the loop partitions and not the physical partitions. The image below shows this process visually



# Tools

The following are provided:-

1. A shell script **multiboot-build.sh** which can create any combination of partitions for BOOT (called BOOT<model>), SYSTEM, and IMAGES on a removable device. This also installs tar archives for the model specific boot partition and for the system partition
2. A script **install\_image.sh** which :-
  - creates a blank image file of requested size and name in a target directory of the same name
  - mounts that image file as a loop device
  - runs raspi\_imager to install the requested image to the loop device
  - runs makedtb to create a composite dtb file for the image in the target directory, applying the overlays and properties defined in init.txt
  - copies the kernel and any initramfs from the mounted boot directory to the target directory
  - copies the original config.txt into that directory for reference
  - creates a file "untreated\_config.txt" containing config.txt lines that have **not** been processed
  - unmounts the image and deletes the loop
  - creates a suggested Grub entry to cut and paste into "/efi/boot/grub.cfg"
  - runs a customisation program that copies in any required additional files for each image (the ones designated with a \*)

The user can abort at each step if anything fails and the preceding steps will be backed out

it uses

- A modified raspi-imager pair **Raspberry\_Pi\_Imager-{x64,arm64}.ApplImage** bundle for arm and intel that allows imaging to a loop mount
- A tool pair for arm and intel **./x64/makedtb|./arm64/makedtb** which scan for and parse the config.txt in an image boot sector and create a merged dtb with all (main) overlays applied and properties (base and overlays) set
- a separate python script **deploy\_custom\_files.py** which detects certain specific images that require custom files and copies them in

### 3. Some utilities

- A shell script **makeimage.sh** to make a separate blank **.img** file of the desired name and size in MB
- A shell script **attachimage.sh** to loop mount an image file (optionally mounting the partitions if there are any)
- A similar script **detachimage.sh**

# Build Instructions

See the specific help files

- [Create Partition Structure.md](#) for instructions on creating the 3 partition structure on one or more USB/microSD drives
- [Install Multiboot Images.md](#) For instructions on adding an image from the Pi repository.
- To follow specific instructions for Fedora, LibreELECT, and Lineage